Platform Development Review

Comprehensive summary of all decisions, architecture decisions, display layouts, data layer strategy, and development roadmap.

PLANNING STATUS

All architecture decisions finalised · 16-phase roadmap confirmed

Prototypes built for all major surfaces · Ready to begin Phase 01

16	9	4	3	5
Phases defined	Display layouts	Pricing tiers	POS integrations	TV platforms

React 18 + TypeScript · .NET 8 Web API · ASP.NET SignalR · RepoDb · Azure SQL

Square · Toast · Clover · Claude AI · Android TV · Samsung Tizen · LG webOS

Contents

1

Executive Summary

Vennu is a cloud-connected digital signage platform built specifically for restaurants, bars, breweries, cafes, and food halls. Where generic signage tools treat hospitality as one vertical among many, Vennu is purpose-built for it — real-time POS integration, restaurant-native display layouts, multilingual support, and a staff mobile app that lets a solo operator update their board in 30 seconds from the kitchen.

1.1 Product Identity

Field	Detail
Name	Vennu (renamed from working title TapBoard)
Tagline	every venue · every menu
Domain	vennu.app (to be registered immediately)
Target Markets	Restaurants · Bars · Breweries · Cafes · QSR · Food Halls
Visual Identity	Playfair Display logo · amber accent · dark theme base
Trademark	USPTO Class 042 — to be filed immediately as first priority action

1.2 Competitive Position

After Phase 10 (POS Integration), Vennu becomes the only platform offering restaurant-specific display layouts, real-time POS 86-ing, multilingual support, and a staff mobile app — all in one product at flat per-venue pricing.

Competitor	Their Pricing	Vennu Advantage
Raydiant	$80+/screen/mo	Flat $89/venue · any hardware · restaurant-native design
ScreenCloud	$20/screen/mo	Real POS integration · restaurant layouts · multilingual
Yodeck	$8/screen/mo	Staff app · POS integration · not Raspberry Pi only
OptiSigns	$10/screen/mo	Real-time 86 from POS · bilingual displays · scheduling
Taplist.io	Beer boards only	Full menu support · cocktails · food · cafes · restaurants

Key differentiator. After Phase 10, Vennu is the only platform covering all five dimensions: restaurant-specific layouts, real-time POS availability updates, multilingual support, staff mobile app, and flat per-venue pricing. No generic competitor comes close on all five simultaneously.

2

Architecture & Technology

2.1 Technology Stack

Layer	Technology	Rationale
Backend	.NET 8 Web API	Modern, performant, native MS developer toolchain
ORM	RepoDb (hybrid ORM)	Faster than EF Core · explicit · works with existing library
Migrations	DbUp (NuGet)	Proven · runs at app startup · transaction-safe scripts
Real-Time	ASP.NET SignalR	Built into .NET · WebSocket · auto-reconnect · group push
Database	Azure SQL + Blob Storage	Scalable · integrated with App Service · CDN for media
Auth	Microsoft Entra ID	Enterprise-ready · multi-tenant · SSO for franchise chains
Frontend	React 18 + TypeScript + Vite	Industry standard · fast HMR · same setup across all SPAs
Mobile	React Native	One codebase for iOS and Android · shared API logic
TV Platforms	Browser · Android TV · Fire TV · Tizen · webOS	Same React SPA wrapped per platform · zero code changes
AI	Claude API (Anthropic)	Descriptions · translation · custom display generation
Payments	Stripe	Subscriptions · trials · HaaS contracts · invoicing
CI/CD	GitHub Actions	Free tier · auto-deploy to Azure on push to main

2.2 Three-Layer Application Architecture

Layer 1 — Super Admin
vennu.app/superadmin

Internal back-office CRM. Dynamic tier management, feature flag matrix, per-venue overrides, MRR dashboard, screen health map. Built around Phase 14 when manual management of ~30+ customers becomes risky.

Layer 2 — Venue Admin

vennu.app/admin

The daily interface for restaurant and bar owners. Menu editing, screen management, scheduling, display configuration. Every feature checked against HasFeatureAsync(venueId, key) — tier-aware from the first line of code.

Layer 3 — Display App

vennu.app/display/{screenId}

The board running on TVs. React SPA connecting via SignalR. Renders the assigned layout and updates in real time. Identical code runs on browser, Android TV, Fire TV, Samsung Tizen, and LG webOS.

3

Display Player

The display player is intentionally simple. Three steps on boot, then it renders whatever the API and SignalR push to it. Layout selection, scheduling, tier logic — all server-side. The player never needs to be rewritten as features are added.

3.1 Boot Sequence

Step	Action	Detail
1	Fetch initial content	GET /api/display/{screenId}/content — sets the initial board state
2	Connect SignalR	JoinScreen({screenId}) — subscribes to ContentUpdated, ThemeUpdated, ItemAvailabilityChan
3	Start heartbeat	POST /heartbeat every 30 seconds — marks screen Online in admin dashboard
4	Register Service Worker	Caches last payload — board keeps showing if internet drops during service

3.2 Layout Switching

The content payload contains a **layout** field. The player selects the corresponding React component and renders it. Adding a new layout in a later phase means adding one React component and one entry in this map — the player's core boot sequence never changes.

3.3 Real-Time Events

Event	Triggered by	Player response
ContentUpdated	Any admin save	Full board re-render within ~200ms
ThemeUpdated	Theme builder save	Colors and fonts update live — no reload
ItemAvailabilityChanged	POS webhook fires	Patches single item in board state — no full re-render
SyncTick(serverTimeMs)	Server clock (16ms)	Synchronises video wall frames across screens

3.4 Offline Resilience

A Service Worker caches the last successful content payload. If internet drops during service, the board keeps showing the last known menu. When connectivity returns SignalR auto-reconnects and pushes

the latest content — the customer never sees a blank or broken screen.

4

Screen Registration

Registration starts simple at MVP and evolves as the platform grows. The screen ID never changes once issued — the display URL is bookmarked once and never touched again.

4.1

Phase 02 — Manual SQL Insert

You manually create screen records for the first 10 customers and hand them a URL. Crude but entirely functional — you're onboarding customers personally at this stage and don't need a UI for it.

4.2

Phase 04 — Self-Serve URL Generation

The admin CMS gets a Register Screen button. The customer generates a URL, types it into a browser on the TV once, and bookmarks it. The screen appears Online within 30 seconds of the first heartbeat ping.

4.3

Phase 08 — 6-Digit Pairing Code

The TV opens vennu.app/pair and displays a 6-digit code. The customer types that code in their admin panel and the screen links instantly. The TV auto-redirects to its display URL. No more typing long URLs on a TV remote. Same UX pattern as Netflix and Roku — customers already understand it.

4.4

Phase 13 — TV App Native Pairing

The Vennu Android TV, Fire TV, Tizen, and webOS apps display the pairing code automatically on first launch. Kiosk mode prevents accidental exit. Auto-launches on TV power-on. App store listings make discovery trivial.

4.5

Phase 11 — HaaS Pre-Registration

Hardware bundles ship with screens already registered to the venue. Correct menu is showing before the box is opened. Customer plugs in, powers on, and the board is live. The card inside reads: 'Your screens are already set up. Plug in, done.'

Key property. The screen ID is permanent across all registration methods. Whether a screen was registered via URL, pairing code, TV app, or HaaS pre-registration — it has exactly one identifier, forever. This makes every other architectural decision about screens simple.

5

Tier System & Monetisation

The tier system is the monetisation engine. Built in Phase 03 — before the customer CMS — so every panel, nav item, and button is tier-aware from day one. Tiers live in the database, not in code, so creating new tiers or moving features between them takes seconds without a code deployment.

5.1 Subscription Tiers

Tier	Price	Screens	Target & Key Features
Starter	$39/mo	2	Generic signage · cafes · QSR · Photo Grid and Classic Diner layouts
Restaurant Starter	$49/mo	1	Solo restaurant operators · meal periods · bilingual · AI translation (1 l
Pro	$89/mo	6	Bars and restaurants · all layouts · happy hour · POS integration · staf
Business	$179/mo	Unlimited	Chains and large venues · everything in Pro plus AI custom builder · n

Annual billing (2 months free, approx. 17% discount) and 14-day free trials with no credit card required are available across all tiers.

5.2 Feature Flag Architecture

Three database entities form the system. The **Feature** table is the catalog of every flag in the platform. The **TierFeature** join table records which flags belong to which tier. The **VenueFeatureOverride** table holds per-venue exceptions with a reason, optional expiry, and admin audit trail.

5.2.1 Resolution Logic

Every controller and UI component calls a single function: **HasFeatureAsync(venueId, featureKey)**. This function checks for a venue-level override first — these always win — and falls back to the tier's default feature set otherwise. The result is fully transparent: any administrator can see exactly what a venue has access to and why.

5.3 Hardware-as-a-Service Bundles

Three HaaS bundles remove the most common barrier to sign-up: sourcing and configuring compatible screens. Hardware ownership transfers at the end of each contract term, after which Stripe automatically transitions the venue to the equivalent software-only tier.

Bundle	Monthly	Term	Includes	Post-Contract
Starter Kit	$89/mo	18-mo	1 screen + Fire Stick	→ $39 Starter

Bar Pack	$159/mo	24-mo	2 screens + mounting + sticks	→ $89 Pro
Full House	$249/mo	36-mo	4 screens + mounting + setup visit	→ $179 Business

Revenue note. HaaS customers generate approximately 8× more lifetime revenue than software-only customers over five years. Full House over 5 years = ~$13,000 gross revenue per venue versus ~$1,600 for software-only Pro.

6

Upgrade Prompt System

The upgrade prompt system turns the product into a self-serve revenue engine. Every prompt follows strict principles to stay informative without becoming annoying or blocking a customer's current workflow.

6.1 Core Principles

Principle	Rule
Show benefit, not tier	'Items sell out → board updates in seconds' — not 'Upgrade to Pro for POS'
Never block workflow	Locked features are visible and informative · never an error or dead end
One prompt per screen	Never more than one upgrade suggestion visible at once · prevents alert fatigue
Always dismissible	Per-session memory · a dismissed hint never reappears in the same session
One click to upgrade	Modal → single CTA → Stripe checkout · maximum two taps from hint to payment
Tier badges only	Small coloured pill (e.g. PRO) beside locked items · informational, never alarming

6.2 The Six Prompt Patterns

6.2.1

Tier Badge

A small coloured pill shown beside any locked feature. Used on nav items, section headers, individual form fields, and layout cards. Color matches the tier required.

6.2.2

Locked Nav Items

Visible in the sidebar at 50% opacity with a tier badge. Clicking opens the upgrade modal, not an error message. Customers can see the feature exists.

6.2.3

Locked Section Preview

A subtly blurred glimpse of what the locked feature looks like in use. One benefit sentence and a soft 'Unlock with Pro' CTA below it.

6.2.4

Inline Feature Hint

An amber-accented card at the bottom of the most relevant panel. For example, a POS hint shown on the menu editor tab. One per panel, dismissible.

6.2.5

Sidebar Nudge

Quietly rotates through locked features at the bottom of the sidebar at 7-second intervals. One sentence and one link per feature. Per-feature dismiss.

6.2.6

Upgrade Modal (Bottom Sheet)

Slides up from the bottom on any locked-feature click. Shows the specific benefit, all features at the required tier, the current tier as context, a single CTA, and a low-guilt 'Maybe later' exit.

7

Display Layouts

Each layout is a React component rendered by the display player based on the layout field in the content payload. All layouts support the theme builder, multi-screen overflow, bilingual rendering, and real-time POS availability updates.

7.1 Layout Catalog

Layout	Tier	Target Venue	Key Characteristics
Photo Grid	All	QSR · Chinese · Vietnamese · Mexican	Photo cards with bestseller ribbons and sold-out overlays · 2×
Classic Diner	All	Diners · cafes · sandwich shops	Text-only on white background · multi-column · high legibility
Neon Chalkboard	Pro	Bars · upscale restaurants	8-layer CSS glow · chalk texture · flicker animation · chalk dra
Split Layout	Pro	Casual mid-range restaurants	Hero photos left · full text menu right · adjustable split ratio
Daily Special Hero	Pro	Any venue with rotating specials	One full-screen featured item · rotating secondary strip · 'Toda
Classic Chalk Drinks	Pro	Cocktail bars	Category pricing in bordered boxes · two-column cocktail list
Tap Strips Board	Pro	Taprooms · craft breweries	Individual strip per tap · rotating hand-lettered fonts · colored
Digital Tap Board	Pro	Modern taprooms	Wood texture · beer glass SVG illustrations · full ABV / IBU /
Custom HTML	Business	Power users and agencies	AI-generated or hand-coded in iframe sandbox · template var

7.2 Theme Builder

Available on Pro and above, with a basic version included on Starter. Includes 5 built-in preset themes, color pickers for every visual element, a glow intensity slider from 0.2× to 2.0×, six venue name font options, and four menu item font options. Every change renders in a live preview. A single Push to All Screens button broadcasts the new theme via SignalR.

8

Data Layer

8.1 ORM — RepoDb

RepoDb was chosen over Entity Framework Core as the data access library. It is a hybrid ORM — more capable than raw Dapper, more explicit than EF Core — and works naturally with an existing data layer library. Operations are implemented as extension methods on IDbConnection, making every database call obvious from reading the code. No change tracking, no lazy loading surprises.

8.2 Schema Migrations — DbUp

DbUp is a .NET NuGet package that tracks which SQL scripts have been run against a database and applies only new ones on each deployment. Scripts are numbered, embedded as assembly resources, and run inside transactions at application startup before traffic is accepted. If a migration fails the API does not start and Azure keeps the previous version running.

8.2.1 Naming Convention

Scripts are numbered sequentially: **001_create_venues.sql**, **002_create_screens.sql**, and so on. Existing scripts are never modified — schema changes always add a new numbered script.

8.2.2 Zero-Downtime Changes

The expand and contract pattern is used for any change touching live data. Step 1: add the new column or table (old code ignores it, new code uses it). Step 2: deploy new code. Step 3: drop the old structure in a later release once nothing references it. Never drop a column in the same release that removes its code references.

8.3 Existing Library Strategy

An existing .NET data layer library will be refactored to use RepoDb under the hood while keeping its existing public API. New Vennu repositories (Screen, Venue, Menu, Feature, Tier) inherit from the refactored base. The DbUp migration runner lives alongside the repositories in the same library.

> **Decision note.** Both a fully custom schema-from-code system (using C# attribute decorations to generate DDL) and a custom DbUp replacement were considered and deliberately rejected. The cost of maintaining and debugging a bespoke migration system outweighs any benefit. DbUp is proven, well-maintained, and solves the problem completely.

9

Development Roadmap

16 phases ordered by architectural dependency. Features are spread across phases so each one ships incremental customer value. The tier system is in Phase 03 — before the Admin CMS — so every feature is tier-aware from day one.

Phase	Title	Duration	Milestone
01	Foundation & Launch Setup	Wks 1–3	Infrastructure running · accounts open
02	Core Backend & Real-Time Engine	Wks 3–7	Screens update in real time
03	Tier System & Feature Flags	Wks 6–9	Monetisation infrastructure live
04	Admin CMS — Core Editing	Wks 8–12	First venue can manage their board
05	Display Layouts — Restaurants & Cafes	Wks 10–14	Can sell to restaurants and cafes
06	Display Layouts — Bars	Wks 13–17	Can sell to bars and upscale restaurants
07	Scheduling Engine	Wks 15–18	Menus run themselves — zero daily effort
08	Tap List Boards — Breweries	Wks 17–21	Can sell to breweries and taprooms
09	Upgrade Prompts & Billing UX	Wks 19–23	Self-serve upgrade funnel generating revenue
10	POS Integration	Mos 5–8	Beats every generic competitor
11	Multilingual Support	Mos 7–9	Ethnic restaurant market unlocked
12	Staff Mobile App	Mos 9–11	Pro tier becomes sticky
13	TV Apps & Platform Distribution	Mos 11–15	App store presence on all TV platforms
14	Super Admin CRM	Mos 13–16	Internal ops at scale (~30 customers)
15	AI Features	Mos 14–18	Premium tier upsell driver live
16	Analytics & Smart Features	Mos 17–22	Data-driven venues · clear ROI proof

10

Development Workflow

10.1 Tooling

Tool	Role
Visual Studio 2022 Community	Primary IDE · build, debug, breakpoints, NuGet packages · free
Claude Code (CLI)	AI coding partner · runs in terminal · reads and writes files directly · no copy-paste
Vite + React 18 + TypeScript	Frontend build tool · fast HMR · same setup for all three SPAs
GitHub	Source control · GitHub Actions for CI/CD to Azure · free private repos
Postman	API testing and webhook simulation · import collections per integration
Figma ($15/mo)	UI design · prototypes from planning phase as starting point

10.2 Claude Code Workflow

Visual Studio stays open as the primary IDE. Claude Code runs in a terminal alongside it (Windows Terminal or VS integrated terminal). Files update in real time as Claude writes them. Visual Studio handles building, debugging, and NuGet packages. Claude Code handles writing new files, refactoring, and fixing errors — without any copy-paste between windows.

10.3 CLAUDE.md — The Project Constitution

A markdown file in the project root that Claude Code reads at the start of every session. Contains the project description, current phase, build commands, naming conventions, and a log of what has been built so far. This file is the single most important document for AI-assisted development on a project this size. Updated as the project evolves.

Recommendation. Setting up a Claude Project before writing the first line of code is strongly recommended. Upload this review document plus a CLAUDE.md file. Every future Claude Code session benefits from that context automatically — no re-briefing required at the start of each development session.

11

Prototypes Built

Seven working JSX prototypes were built during the planning phase. These capture visual, interaction, and layout decisions for reference when building production components. They are not production-ready code — they are high-fidelity references.

File	What It Covers
Vennu.jsx	Full admin CMS — menu editor, scheduling, displays, Quick Update mobile view, bilingual
VennuTapList.jsx	Three tap list board styles side by side — Classic Chalk, Tap Strips, Digital Board
NeonChalkboard.jsx	Flagship Neon Chalkboard layout with 4 themes — glow, flicker, chalk draw-in animation
ThemeBuilder.jsx	Full theme editor — color pickers, glow slider, font selectors, live preview
PhotoMenuBuilder.jsx	Photo Grid layout with multi-screen overflow visualizer
VennuSuperAdmin.jsx	Super Admin CRM — tier manager, feature matrix, venue CRM, screen health map
VennuUpgrade.jsx	All six upgrade prompt patterns in a simulated Restaurant Starter venue account

Reference the visual and interaction decisions in these files when implementing production components from Phase 04 onwards. CSS-in-JS animation patterns, color values, and layout structures can be used as starting points.

12

Next Steps

Concrete actions ordered chronologically. Securing the brand and domain are the single most time-sensitive items — these cannot wait.

12.1 This Week

- Set up Claude Project — upload this PDF and prototype JSX files as context
- Register vennu.app domain immediately — approximately $12/yr, cannot be delayed
- File Vennu trademark — USPTO Class 042 — approximately $250–350 plus attorney
- Create Azure subscription — resource group, App Service B1, Azure SQL Basic
- Initialise GitHub repository — .NET 8 API and Vite React monorepo, connect Actions

12.2 Weeks 2–4

- Refactor existing data library — RepoDb foundation, DbUp migrations, base repositories
- Scaffold all core data models — Venue, Screen, Menu, Feature, SubscriptionTier, ScreenPairingCode
- Build SignalR hub — JoinScreen and ContentUpdated events, test with two browser tabs
- Set up Stripe — create product and price objects for all four tiers, monthly and annual

12.3 Month 2

- Build HasFeatureAsync service — used by every controller from this point forward
- Build admin menu editor — inline editing, 86 toggle, Quick Update mobile view, tier-aware
- Build Photo Grid layout — first display layout, real-time SignalR, offline Service Worker
- Sign first beta customer — one venue, one screen, no charge while testing

12.4 Month 3

- Build Classic Diner layout — second layout, unlocks cafes and diners as a market
- Build Neon Chalkboard — flagship bar layout with full theme builder
- Launch scheduling engine — meal periods and happy hour, IHostedService evaluator
- Sign first paying customer — validate willingness to pay before building further

12.5 Months 4–5

- Build all three tap list board styles — unlocks breweries as a market
- Build upgrade prompt system — all six patterns, Stripe self-serve checkout
- Begin Square POS integration — OAuth connect, catalog sync, inventory webhook
- Pairing code registration flow — vennu.app/pair, 6-digit code, admin claim endpoint

Priority action. The single highest-impact action before any code is written: set up the Claude Project with this document uploaded. Every future session starts with full context — architecture, decisions, conventions, current phase — without any manual briefing. Ten minutes of setup now saves hours across the entire build.